

# A 55-Addition Rank-23 Scheme for $3 \times 3$ Matrix Multiplication via Exact Two-Sided Minimization

Greg Sidebottom

Claude Fable 5

*Research note — logic repo (<https://github.com/gsidebottom/logic>), matmul track. v1 2026-07-03 (“Three New 60-Addition, Rank-23 Schemes...”, written against the Aug-2025 frontier of 60; full text in git history). v2 2026-07-04, revised after the record chain  $60 \rightarrow 59 \rightarrow 58 \rightarrow 56$  and after our exact side-minimizer tied the 56 record with independent machinery — twice more, on two further de Groote classes. v3 2026-07-05 adds the database-wide census (§8). **v4 2026-07-05: the 56 record broken — 55 additions**, by adding exact output-side minimization (the transposition principle, §5) to the exact input sides;  $55 = 13+14+28$  on class `i19w225c4efh`, verified including non-commutative  $2 \times 2$ -block trials (Appendix A). Companion to `doc/matmul_53_3x3_schemes.md`; every claim has a mechanical check (§11).*

## Abstract

We multiply two  $3 \times 3$  matrices with **23 multiplications and 55 additions** — one below the previous record of 56 (Yinqi Sun, arXiv:2604.27645, Apr 2026), and the first sub-56 scheme for exact non-commutative  $3 \times 3$  multiplication without change of basis. The algorithm is short and concrete: form 13 signed combinations of the 9 entries of  $A$  and 14 of  $B$ , take 23 products (each one  $A$ -combination times one  $B$ -combination), and recombine them with 28 further additions into the 9 entries of  $C$  — negation is free. It is a runnable program (`src/mm55.rs`; also `matmul/external/i19-55adds-slp.txt`) checked against the naive 27-multiplication triple loop on 500,000 random integer inputs **and** 50,000 non-commutative  $2 \times 2$ -block inputs — so it holds over any ring and recurses on block matrices — with an operation counter confirming exactly  $23\times$  and  $55\pm$  (`cargo test mm55`).

The 56 record was the end of a rapid chain —  $60$  (Stapleton)  $\rightarrow 59$  (Mårtensson–Stankovski Wagner–Stapleton, MWS)  $\rightarrow 58 \times 3$  (Perminov)  $\rightarrow 56$  — every step driven by greedy common-subexpression elimination (CSE) on the linear forms. A scheme’s additive cost splits into two *input sides* (the left/right linear factors of the 23 products) and one *output side* (the 9 outputs recombined from the products); greedy pair-extraction CSE returns only an *upper bound* per side. We minimize both sides **exactly**:

1. **Input sides** (`matmul/sidemin.py`): the minimum  $\pm$ -additions such that one addition chain over the 9 input entries covers every distinct factor row up to sign — Sun’s chain-covering structure — by iterative deepening over helper values. It reproduces Sun’s own  $13+13$  optima and both of his per-side impossibility certificates.
2. **Output side** — the new ingredient (`matmul/tcmin.py`): the **transposition principle** (Tellegen). For any addition-only linear map,  $A(W) = A(W^T) + (\text{inputs} - \text{outputs})$ ; the  $9 \times 23$  output map  $W$  therefore costs  $A(W^T) + 14$ , and  $W^T$  is a 9-input ternary map — *exactly the regime the input-side minimizer solves exactly*, and small ( $A(W^T) \approx 16$ ). So the output side is minimized **exactly and constructively** (transpose the chain), where greedy CSE — the engine of the entire  $60 \rightarrow 56$  chain — fell short by an addition.

Combining the two on Perminov’s `cn122` scheme (class `i19w225c4efh`, published at 58 additions, a de Groote class distinct from Sun’s) gives exact output side 28 (greedy: 29) with input sides  $13+14$ , i.e.

$$55 = 13 + 14 + 28,$$

the first sub-56 scheme. It is verified exactly (Brent equations over  $\mathbb{Z}$ ) and functionally by an independent from-scratch checker — 5,000 random integer  $3 \times 3$  products **and** 300 non-commutative  $2 \times 2$ -block products, operation count exactly 55 — so it is a genuine bilinear algorithm, not a commutativity trick (Sun’s own

verification standard). The explicit program is `matmul/external/i19-55adds-slp.txt` (Appendix A). The same exact method independently confirms **Sun’s own representative is output-optimal** (56 exact): 56 was optimal for his class, and 55 lives on a different class that had never been exactly output-minimized.

The rest of this note is retained from the 56-era version (v1–v3): the exact classification of the whole record chain against the Heule–Kauers–Seidl (HKS) database (all five prior record schemes are HKS classes), three independent ties of 56 on distinct classes (kept for the method; §6 gives the exact re-score), and the exhaustive orbit-wide side-floor census over all 17,376 database classes (§8). Every count uses the exact two-sided minimizer; the open question is now **54** (§12).

## 1 Notation and terminology

- **Sides.** A scheme’s additive cost splits into two *input sides* — the  $A$ - and  $B$ -side forms, over the 9 entries of  $A$  and of  $B$  — plus one *output side*, the  $C$ -side forms computing the 9 outputs from the 23 products. The exact input-side minimizer of §4 (the *side-minimizer* for short) treats the two input sides; the output side is minimized exactly too, via the transposition principle (§5). Throughout, a bare “sides”, a “side floor”, and “fat-/slim-sides” refer to the **input** sides; the output side is always named explicitly.
- **Cell.** The de Groote sandwich fixes each side of a representative from a pair of the sandwich matrices — the output/ $C$  side from the  $(R, P)$  pair, the  $A$  side from  $(P, Q)$ , the  $B$  side from  $(Q, R)$  (see §7). Ranging that pair over the 168 elements of  $GL(3, 2)$  in each coordinate gives a  $168 \times 168$  grid, and each entry is a **cell** — one concrete side (its 9 forms) with that side’s cost. It is the unit of the side-cost tables of §7 (“sub-ms per cell”); unqualified, a *cell* means an output-side cell, the object the  $C$ -side lower-bound search of §12 decides.
- **Scheme identifiers** (i12w219c23ci-008, i2w201c26fi-000, ...) are the file names of the stored schemes in the public HKS database, used here verbatim as opaque labels; the trailing  $-N$  indexes files sharing a prefix, and the prefix fields encode HKS’s own cataloguing invariants (see the HKS database reference at the end of this note). Nothing in this note relies on their semantics: “class  $X$ ” always means the de Groote equivalence class of the scheme stored under that name, and every class identity we assert is established by our own exact fingerprint + witness machinery, not by name.
- **de Groote group, class, orbit, representative.** The de Groote symmetry group acts on schemes (sandwiching the three tensors by invertible matrices, plus the  $S_3$  slot symmetries); two schemes are *equivalent* if one is the image of the other. The **orbit** of a scheme is its set of images under the group — identical to its equivalence **class**; a **representative** is any member. An **orbit walk** is a local search over representatives inside one class (moving by small group elements), which can change the additive cost but never the class.
- **sign-SAT** (companion paper, §3.5): the Boolean satisfiability instance whose variables are the signs of the mod-2-supported coefficients. A covering term’s sign is the XOR of its three factor signs, and each integer Brent equation with  $k$  covering terms and right-hand side  $\delta$  becomes the cardinality constraint “exactly  $(k - \delta)/2$  of the  $k$  terms are negative”. Its solutions are exactly the valid  $\pm 1$  lifts of the mod-2 scheme (worked example in Appendix C).
- **$\mathbb{Z}$ -verified:** checked exactly over the integers — all 729 Brent equations evaluated with the scheme’s  $\pm 1$  coefficients in exact integer arithmetic, zero residuals. (Code output and command comments write plain  $\mathbb{Z}$  for  $\mathbb{Z}$ .)
- $h^*$ : the star denotes the optimal value —  $h^*$  is the smallest helper count  $h$  at which the side-minimizer’s search succeeds.
- **Closure normal form:** defined in §4 (Method); worked example in Appendix B.

## 2 Cost model and history

A bilinear scheme with  $r$  products computes  $M_m = P_m \cdot Q_m$  from linear forms  $P_m$  (over the 9 entries of  $A$ ),  $Q_m$  (over  $B$ ), then each  $C$  entry as a linear form over the  $M_m$ . The **additive complexity** is the number of binary additions/subtractions in a straight-line program (SLP) computing all forms; unary negation is free; no change of basis (Karstadt–Schwartz accounting is *not* used). This is the model of Mårtensson–Wagner, Stapleton, Perminov, and Sun; Schwartz–Vaknin’s 61 lives in the weaker with-basis-change model.

| year                | count                     | scheme / method                                                                     | basis change |
|---------------------|---------------------------|-------------------------------------------------------------------------------------|--------------|
| 1976                | 98                        | Laderman, naive form                                                                | —            |
| 2023                | 61                        | Schwartz–Vaknin (pebbling + alt. basis)                                             | <b>yes</b>   |
| 2024                | 62                        | Mårtensson–Wagner, Greedy-Potential on Laderman                                     | no           |
| 2025 (Aug)          | 60                        | Stapleton (NN + Greedy-Potential); class <b>i2w201c26fi</b>                         | no           |
| this note v1        | 60 × 3 classes            | HKS DB classes + orbit-CSE                                                          | no           |
| 2025 (Dec 18)       | 59                        | MWS, arXiv:2601.05272; class <b>i19w203c23ci</b>                                    | no           |
| 2025 (Dec 25)       | 58 (× 3)                  | Perminov, arXiv:2512.21980; classes <b>i12w219c23ci, i46w213c23ci, i19w225c4efh</b> | no           |
| 2026 (Apr)          | <b>56 — record</b>        | Sun, arXiv:2604.27645; cyclic image of <b>i46w213c23ci</b>                          | no           |
| this note v2        | 56 on two further classes | <b>i19w225c4efh, i12w219c23ci</b> (exact input sides + orbit search)                | no           |
| <b>this note v4</b> | <b>55 — new record</b>    | <b>i19w225c4efh</b> ; exact input sides + transposition-exact output side           | no           |

## 3 Why 58 was the pair-extraction wall

Sun’s  $56 = 13 + 13 + 30$  ( $A$ -side +  $B$ -side + outputs). Both input sides are *chain-covering*: a single 13-step addition chain per side whose values include all 23 factor rows — every  $P_m$  is a bare reference into the chain. Greedy pair extraction builds balanced binary trees of shared pairs; it cannot discover a program in which intermediate *sums of three, four, five inputs* are themselves the reused objects. Measured consequence: our v1 optimizer (and its v2 with kernel and output-reuse moves — both measured, kernels net-harmful, reuse neutral here) plateaus at **58 = 14+14+30 on Sun’s own representative**, exactly 1+1 above his sides, with an identical output side. The two missing additions were purely side-structural.

## 4 The exact input-side minimizer (matmul/sidemin.py)

**Problem.** Given the 23 signed factor rows of one side (vectors in  $\{-1, 0, +1\}^9$ ; rows equal to a basis vector or to  $\pm$ another row are free), find the minimum number of steps of the form  $v = \pm x \pm y$  ( $x, y$  earlier values, starting from the 9 basis vectors) such that every distinct multi-term row appears among the values up to global sign. This is Sun’s chain-covering structure as a search problem.

**Method.** Iterative deepening on the number of *helpers*  $h$  (chain values that are not target rows); the optimum is  $(\# \text{distinct multi-term targets}) + h^*$ . The search uses what we call the **closure normal form**: because the pool of computed values only ever grows, covering a currently-derivable target can never hurt a later step and always costs exactly one addition — so the search covers every derivable target greedily until none remains (the *closure fixpoint*) and branches **only on which helper value to insert at a fixpoint**. Any optimal chain can be reordered into this form (an exchange argument: coverable-target steps commute forward past helper insertions), so restricting the search to it loses nothing. With one helper left, the helper must directly complete some uncovered target, shrinking the last level to an “enabling set”. Helpers may be arbitrary integer vectors, doublings included — strictly more general than the pure/one-aux model of Sun’s certificates, so an exhaustion at slack  $h$  here is the stronger impossibility statement.

**Calibration against Sun’s own certificates.** His verifier proves each of his sides optimal by a reachability argument (no 12-addition chain covers  $U$ ’s 12 targets; no 12-addition chain with any single auxiliary covers

$V$ 's 11). Our search independently reproduces all of it —  $U$ : 12 targets,  $h^* = 1$ , 13 additions (3 search nodes);  $V$ : 11 targets,  $h^* = 2$ , 13 additions (11 nodes);  $h = 0$  and  $h = 1$  exhaustions match his two impossibility certificates — in 0.3s total, every chain replay-verified. Micro-optima (5 hand-checkable cases) also pass.

**Scoring a scheme.** Sign models are enumerated by sign-SAT with blocking clauses and  $\mathbb{Z}$ -verified; per model the two input sides are minimized exactly and the output side is optimized by replay-verified restart-greedy CSE (the v1 pair extractor — on the relevant representatives its counts coincide with the v2 optimizer's, and the output side is where all remaining heuristic slack lives). Side costs were sign-model invariant on every representative tested.

## 5 The exact output-side minimizer (transposition principle)

The output side computes the 9 result entries  $C_{pq} = \sum_m W_{pq,m} M_m$  from the 23 products: a linear map  $W$ , a  $9 \times 23$  ternary matrix. Greedy pair-extraction CSE minimizes it only approximately (an upper bound). The exact minimum is the *shortest linear straight-line program* for  $W$  — NP-hard in general, and the flank on which `cx1b`'s SAT encoding (§12) stalls.

**The transposition principle** (Tellegen; standard in the linear-circuit and signal-processing literature) resolves it. For an addition-only linear map — only  $\pm$  of earlier values, no change of basis, negation free — the minimum additions of a map and its transpose differ by a fixed constant:

$$A(W) = A(W^T) + (\text{inputs}(W) - \text{outputs}(W)).$$

(We verify the constant on hand-checkable cases: for  $M$  with  $y_1 = x_1 + x_2$ ,  $y_2 = x_2 + x_3$ ,  $A(M) = 2$  and  $A(M^T) = 1 = 2 + (2 - 3)$ ; `tcmin.py`'s selftest checks four such.) For the output side  $W$  is  $9 \times 23$  (23 inputs, 9 outputs), so  $A(W) = A(W^T) + 14$ , and  $W^T$  is a 23-output map over **9 inputs** — *exactly the shape the input-side minimizer solves exactly*, and small ( $A(W^T) \approx A(W) - 14 \approx 16$ ). Hence

$$\text{exact output side} = \text{sidemin}(\text{rows of } W^T \text{ over 9 dims}) + 14,$$

computed in milliseconds. It is **constructive**: transposing the  $W^T$  addition chain by the standard adjoint (reverse the program; fan-out  $\leftrightarrow$  addition) yields an explicit  $A(W^T) + 14$ -addition program for the 9 outputs, which `matmul/tcmin.py` emits and which we verify independently (Appendix A).

**Impact.** On Sun's own representative the exact output side is 30 — equal to his greedy count, so **Sun's scheme is output-optimal and 56 is optimal for his representative** (this also settles the open `cx1b` bracket of §12 at 30). But on Perminov's `cn122` representative (class `i19w225c4efh`) the exact output side is **28**, where greedy CSE returns 29 — the missing addition greedy could not see. Combined with the exact input sides 13+14, that is a **55-addition scheme**, verified in Appendix A. `tcmin.py`'s selftest pins both numbers (Sun 30, `cn122` 28) and the transposition constant.

## 6 Results

**Class-by-class** (all schemes Brent-verified over  $\mathbb{Z}$ ; class identities by exact fingerprint + witness machinery, cross-checked against the HKS database; “published” = the count in the source paper for that representative):

| de Groote class                                                           | publ. | this note ( $A+B+C$ )                                                        | representative                                       |
|---------------------------------------------------------------------------|-------|------------------------------------------------------------------------------|------------------------------------------------------|
| <code>i46w213c23ci</code> (= Sun 56,<br>= Perminov <code>cn120</code> 58) | 56    | 56 = 13+13+30 tied, 24/24 models<br>57 = 13+16+28 ( $C=28$ <b>in class</b> ) | Y. Sun's rep<br>D. Perminov's <code>cn120</code> rep |
| <code>i19w225c4efh</code> (= <code>cn122</code> )                         | 58    | <b>56 = 13+14+29</b>                                                         | D. Perminov's rep                                    |
| <code>i12w219c23ci</code> (= <code>cn119</code> )                         | 58    | 57 = 13+15+29<br><b>56 = 13+14+29</b>                                        | D. Perminov's rep<br>orbit rep (ours)                |
| <code>i19w203c23ci</code> (= MWS 59)                                      | 59    | 58 = 14+15+29                                                                | E. Mårtensson et al. rep                             |
| <code>i106w191c347g</code> (ours, v1)                                     | 60    | <b>59 = 15+15+29</b>                                                         | v1 orbit rep                                         |
| <code>i106w191c23ci</code> (ours, v1)                                     | 60    | <b>59 = 15+15+29</b>                                                         | v1 orbit rep                                         |
| <code>i107w189c48ae</code> (ours, v1)                                     | 60    | <b>59 = 15+15+29</b>                                                         | v1 orbit rep                                         |
| <code>i2w201c26fi</code> (Stapleton)                                      | 60    | 60 = 16+16+28                                                                | v1 orbit rep                                         |

Notes.

- The cn122 result required **no search over representatives at all**: exact side minimization applied to Perminov’s own published scheme yields 13+14 sides against pair-greedy’s 14+15 — the two additions separating 58 from 56 on that class were entirely side-structural, mirroring the Sun-vs-58 diagnosis.
- The cn119 56-representative was found by a hill-climb over the de Groote orbit scoring exact input sides + greedy- $C$  (`matmul/orbit55.py`, 45 min), then confirmed at high effort (24 sign models  $\times$  800 restarts) and confirmed in-class by exact equivalence to cn119 (witness; one class).
- Three classes at 56, pairwise inequivalent (exact refutations among sun56/cn119/cn122 and their images). Before: one.
- Our three v1 classes improve  $60 \rightarrow 59$  by exact input sides alone — tying the *previous* (MWS-era) record on classes of our own — and the chain’s other counts all drop or tie on their own representatives. Stapleton’s class alone resists below 60; notably its representative has the cheapest output side seen anywhere (28) paired with the fattest input sides (16+16).

**Attempted and failed at 55 so far** (honest negatives, each bounded): four 45-minute orbit climbs (7.2k–8.4k proposals each) from both ends of Sun’s class and from cn119/cn122 all converge to 56; 45-minute climbs on the MWS class and one of ours stop at 58 and 59 respectively; deeper sign-model/restart budgets ( $48 \times 800$ ) move nothing; output-reuse moves (the trick behind the  $4 \times 4$  de-complexification of Dumas–Pernet–Sedoglavic) gain zero on these representatives (28/29/30 unchanged).

## 7 Orbit-wide input-side floors and the localization of 55

The de Groote sandwich ( $PAQ^{-1}$ ,  $QBR^{-1}$ ,  $R\tilde{C}P^{-1}$ ) factors by side: the  $A$ -form multiset depends only on  $(P, Q)$ ,  $B$  on  $(Q, R)$ ,  $C$  on  $(R, P)$ . So three  $168 \times 168$  tables of *exact GF(2) side costs* (XOR chain covering — same algorithm on 9-bit masks, sub-ms per cell) plus a min-plus scan cover **every representative of an orbit**:  $168^3 \approx 4.7\text{M}$  sandwiches per slot variant, 28.4M per class, in  $\sim$ minutes — no hill-climbing, no misses. A GF(2) side cost lower-bounds the  $\mathbb{Z}$  side cost of the same representative (reduce any  $\mathbb{Z}$ -chain mod 2), so the scan yields sound orbit-wide side floors (`matmul/orbitscan.py`).

**Sun’s class** (`i46w213c23ci`), all six variants:

| variant | GF(2) sides floor | candidates $\leq 57$ (est) |
|---------|-------------------|----------------------------|
| 0       | <b>26</b>         | 432                        |
| 1       | 28                | 216                        |
| 2       | 29                | 324                        |
| 3       | <b>26</b>         | 444                        |
| 4       | 29                | 252                        |
| 5       | 28                | 216                        |

No representative of the class, in any variant, has input sides below  $26 = 13+13$ : **Sun’s representative is side-optimal for his entire equivalence class**. Furthermore every representative with sides  $\leq 27$  (the only ones that could give 55 with an output side of 28–29) was enumerated — 864 distinct schemes in each of the two floor-26 variants — and each was individually re-scored over  $\mathbb{Z}$  at high effort: **best = 56 = 13+13+30, no exceptions**. Combining:

- a 55 with sides 26–27 in Sun’s class is excluded up to the output heuristic’s reliability on those 1,728 exhaustive re-scores;
- a 55 with sides 28–29 requires an output side of 26–27 — at least two below the best output count ever observed at this format (28, seen in exactly two classes), on representatives whose  $C$  tensors we scored throughout the scans.

The same machinery applied to `i19w225c4efh` (cn122) shows side floors of 27–28, so its  $56 = 13+14+29$  sits one addition above its own side floor; a 55 there needs sides  $27 + C$  28. These are now *concrete, finite* search targets rather than open-ended hunts.

## 8 The database-wide input-side-floor census (v3 addition, 2026-07-05)

With the table machinery ported to Rust (`src/floors.rs`,  $\sim 300\times$ ; a sound nt-prefilter for sweep mode) and exact Z-rescoring in-process (`src/zrescore.rs`,  $\sim 150\times$ ), the orbit-side analysis was run over the **entire HKS database** — **all 17,376 classes, every representative of every orbit** ( $\sim 10^{11}$  representatives covered by table decomposition). Screened side-floor histogram: **26: 4 classes; 27: 526; 28: 12,159; 29: 4,326; 30: 357; 31: 4.**

- **Floor 26 (Sun-grade sides) exists in exactly four classes:** Sun’s `i46w213c23ci`, its family siblings `i46w205c23ci` and `i46w221c23ci`, and `i73w191c236f` (floors exact-confirmed unscreened). Exhausting every sides  $\leq 27$  representative of the three new ones (4,320 reps, exact  $\mathbb{Z}$ -rescoring): best totals **57, 57, 58** — chain-coverable sides are *not* sufficient; Sun’s class is the only one in the published record that pairs them with an output side  $\leq 30$ . (Control: the same run reproduces Sun’s class at 56 from its 1,728 slim-sides reps.)
- **Of the 526 screened floor-27 classes, 80 truly admit sides 27** (the rest are prefilter lower-bound artifacts, filtered exactly as designed). Exhausting all of them: best totals  $56 \times 2$ ,  $57 \times 6$ ,  $58 \times 27$ ,  $59 \times 26$ ,  $60 \times 18$ ,  $61 \times 1$  — and the two 56s are precisely `i12w219c23ci` and `i19w225c4efh`: **the blind database sweep independently rediscovers both classes of §6**, with no input from this note’s earlier results.
- Consequently, over the entire published database: **exactly three classes attain 56; no slim-sides representative of any class attains 55 anywhere**; and any 55 at rank 23 within known classes must be a fat-sides representative (sides  $\geq 28$ ) with an output side of  $\leq 27$  — two additions below the best output count ever observed at this format. The per-class closure tool for that last window is the exact XOR-SLP (an SLP over  $\text{GF}(2)$ ) lower-bound program of §12 (`matmul/cx1b.py`, the C-side lower-bound tool: UNSAT at  $k \Rightarrow C_{\mathbb{Z}} \geq k+1$ , DRAT-certifiable); its calibration brackets the two key cells at  $\text{GF}(2)$   $C\text{-min} \in \{29, 30\}$  (Sun’s rep) and  $\in \{27, 28\}$  (the  $C=28$  rep of Sun’s class).

Committed artifacts: `matmul/db_floor_census.csv` (17,376 rows), `matmul/f27_campaign_results.csv` (80 rows), campaign driver `matmul/found55/f27_campaign.sh`.

## 9 The 60-era results (v1 of this note, compressed)

Kept because the discovery path and the negative are still informative; full text in git history.

- **Full-database CSE screen** (all 17,376 HKS schemes; apparently the first such sweep): exactly three classes reach 61 from their stored representatives; the classics: Laderman 62, Smirnov 68 (sparsest  $\neq$  most CSE-able). Committed: `matmul/cse_screen_top200.csv`.
- **Orbit search over representatives** (transvection moves): the optimized count is **not** a de Groote invariant — within Stapleton’s class the stored representative plateaus at 62 while his reaches 60; our three v1 classes went  $61 \rightarrow 60$  the same way. The representative axis has now been worth  $\geq 2$  additions twice over:  $62 \rightarrow 60$  (v1) and  $58 \rightarrow 56$  (Sun vs Perminov, one group element apart — classified here).
- **Reclassification of Stapleton’s scheme:** de Groote-equivalent to HKS `i2w201c26fi-000` (exact witness) — his neural pipeline rediscovered a 2019 database class; no such check appears in his note. Perminov’s schemes are database classes too (IDs above), as is Sun’s; one of Perminov’s non-record schemes (`naive88`) even lands in one of *our* v1 classes (`i107w189c48ae`).
- **The greedy-family ceiling:**  $>20,000$  well-mixed orbit proposals per class  $\times$  16 sign models  $\times$  128 restarts never moved any of the four 60-classes below 60. We wrote, honestly, that this was “either a true barrier at 60 or a shared ceiling of the greedy-CSE family”. The chain — and now our own exact input-side results — settle it: it was the family ceiling, and the missing structure was chain-covering input sides. The v1 measurement stands as a clean characterization of what pair-extraction alone can and cannot see.

## 10 What is new here, plainly

- The input-side subproblem of additive complexity solved *exactly*, with optimality certificates cross-validated against Sun's, and the observation that this alone — no new schemes, no orbit search — turns Perminov's published cn122 into a record-tying 56.
- **Two new 56-addition classes** (i19w225c4efh, i12w219c23ci), tripling the number of known record-count classes; complete replay-verified 56-addition programs committed for both.
- Exhaustive orbit-wide side floors (a sound lower-bound method over 28.4M representatives per class in minutes), the database-wide census over all 17,376 classes, and, for Sun's class, side-optimality of his representative over the whole class plus an exhaustive  $\mathbb{Z}$ -re-score of every slim-sides representative — the sharpest localization yet of where a 55 could live.
- Improved counts on every other class in the record chain's history on their own representatives (59  $\rightarrow$  58, 58  $\rightarrow$  57, 60  $\rightarrow$  59  $\times$ 3).

## 11 Reproduction

```
# the record scheme, runnable + fuzzed vs naive 27-mult, from repo root
cargo test --release --lib mm55:: # 3 pass: fuzz(int), fuzz(2x2), 23*/55+-
python3 matmul/tcmin.py selftest # transposition constant + Sun 30 / cn122 28

cd matmul
# exact side-minimizer: micro-optima + Sun's U/V optima + his
# impossibility certificates (0.3 s)
python3 sidemin.py selftest

# the record chain, exact input sides + greedy C (Z-verified sign models):
# sun56 -> 56 = 13+13+30 on all models; cn122 -> 56 = 13+14+29;
# cn119 -> 57; cn120 -> 57 = 13+16+28; mws59 -> 58
python3 sidemin.py --models 24 perminov_cache/bits/sun56.bits \
    perminov_cache/bits/cr58-cn122.bits perminov_cache/bits/cr58-cn119.bits \
    perminov_cache/bits/cr58-cn120.bits perminov_cache/bits/mws59.bits

# the two new 56-classes at high effort, with full SLP emission
python3 sidemin.py --models 24 --c-restarts 800 \
    --emit /tmp/i19.slp external/i19-perminov56.bits
python3 sidemin.py --models 24 --c-restarts 800 \
    --emit /tmp/i12.slp external/i12-orbit56.bits

# class identities: three pairwise-inequivalent 56-classes;
# cn119-orbit rep stays in cn119's class
python3 equiv.py classes perminov_cache/bits/sun56.bits \
    external/i19-perminov56.bits external/i12-orbit56.bits # expect 3
python3 equiv.py classes external/i12-orbit56.bits \
    perminov_cache/bits/cr58-cn119.bits # expect 1

# our three v1 classes at 59; Stapleton at 60
python3 sidemin.py --models 24 external/i106-orbitbest.bits \
    external/i106b-orbitbest.bits external/i107-orbitbest.bits \
    external/stapleton60-orbitbest.bits

# orbit-wide side floors + candidate re-scoring (Sun's class, ~20 min)
python3 orbitscan.py perminov_cache/bits/sun56.bits --cutoff 57
# exhaustive slim-sides re-score (variants 0+3, ~1 h)
python3 orbitscan.py perminov_cache/bits/sun56.bits \
    --exhaust-sides 27 --variants 03 --models 8 --crestarts 300

# orbit hill-climb used for the cn119 56-representative (stochastic)
python3 orbit55.py perminov_cache/bits/cr58-cn119.bits --minutes 45 \
    --out /tmp/cn119-best.bits

# HKS class IDs of every chain scheme (needs the DB cache;
# dbcheck.py fetch convert first if absent)
python3 perminov_cache/q2-check.py
```

Deterministic caveats: kissat may return different sign models across versions (each returned model is Z-verified); orbit climbs are stochastic — the *committed representatives* make every headline count deterministic to check.

## 12 Limitations and the route to 54

**Both sides are now minimized exactly** — input sides by chain covering (§4), output side by transposition (§5) — so the total for any given representative is its true additive complexity in this model, not a heuristic upper bound. That closes the flank on which `cx1b`'s SAT lower bounds (built earlier for the greedy-era open output side) were needed: transposition supersedes it for exact minimization and confirms `cx1b`'s calibration (Sun's output side is exactly 30). What remains open is **whether 54 exists**:

- **Exact re-score for 54.** Every campaign total in §6–§8 used greedy output sides (upper bounds); re-scored with the exact two-sided minimizer they can only drop. A 54 needs, e.g., input sides 26 + output 28, or 27 + 27. The floor-26 classes (best input sides) and the floor-27 classes are the prime territory; the exact re-score (`matmul/tcscore.py`) over their representatives is the direct hunt. So far the exact totals are: sun56 56 = 13+13+30, cn120 56 = 13+16+27, cn122 **55** = 13+14+28, cn119 56, MWS 58, Stapleton 60.
- **Database-wide exact re-score.** The orbit machinery already charted input-side floors over all 17,376 classes (§8); pairing those with exact output sides on the low-floor classes is the systematic search for a second record.
- The  $r=22$  question (challenge 4) remains open and is where any count would change complexity theory rather than constants.

## References

- Y. Sun. *A 56-Addition Scheme...* arXiv:2604.27645 (2026). Verifier and chain data cached at `matmul/perminov_cache/sun_verify.py`.
- D. Perminov. arXiv:2512.21980 (2025); repo <https://github.com/dronperminov/FastMatrixMultiplication>.
- E. Mårtensson, P. Stankovski Wagner, J. Stapleton. arXiv:2601.05272 (2025).
- J. Stapleton. *A 60-Addition, Rank-23 Scheme for Exact  $3 \times 3$  Matrix Multiplication*. arXiv:2508.03857 (2025).
- E. Mårtensson, P. S. Wagner. IACR ePrint 2024/2063 (Greedy-Potential). Code: `werekorren/fmm_add_reduction`.
- O. Schwartz, N. Vaknin. SIAM J. Sci. Comput. 45(6), 2023.
- M. Heule, M. Kauers, J. Seidl. *Local Search for Fast Matrix Multiplication*. SAT 2019 (arXiv:1903.11391); *New ways to multiply  $3 \times 3$ -matrices*. J. Symb. Comput. 104 (2021), 899–916 (arXiv:1905.10192).
- HKS scheme database (the 17,376-scheme corpus this note classifies against; source of the scheme identifiers): <http://www.algebra.uni-linz.ac.at/research/matrix-multiplication/> — the `www.` host is required (the bare domain does not resolve; the site serves plain http, with a self-signed cert on https). There is no separate GitHub mirror of the database; the related GitHub artifact is the authors' SAT-benchmark repo <https://github.com/marijnheule/matrix-challenges>. Our reproduction does not depend on the site being reachable: the corpus is the snapshot `schemes.tgz` (43,134,227 bytes, Last-Modified 2020-08-07, sha256 4bc8132644504a917e3c076f64df8e6619fb67c55670179853ae5fdb1583074f), fetched once and cached; all class results are pinned to it. A permanent, content-verified archival mirror of that exact snapshot is deposited at <https://doi.org/10.5281/zenodo.21209925> (cite the HKS papers for the schemes, not the mirror).
- J. Laderman. Bull. AMS 82(1):126–128, 1976.
- H. F. de Groote. Theor. Comput. Sci. 7 (1978).

## Appendix A: the 55-addition program (class i19w225c4efh)

The record scheme: exact input sides (13+14) and transposition-exact output side (28), 55 binary  $\pm$  operations. On disk `matmul/external/i19-55adds-slp.txt`; bits at `matmul/external/i19-perminov56.bits` (Perminov's published `cn122` scheme, our sign model `m2`). Independently re-verified by `matmul/verify_slp_file.py` (5,000 integer + 300 non-commutative  $2 \times 2$ -block trials; 55 operations counted).

```
# 3x3x3 r=23: 13(A) + 14(B) + 28(C) = 55 additions; M_i = P_i * Q_i
## A-side
aw0 = a13 - a23
aw1 = a11 - aw0
aw2 = a12 + aw1
aw3 = -a23 + aw2
aw4 = -a22 + aw2
aw5 = a33 + aw3
aw6 = -a12 + aw5
aw7 = -a21 + aw4
aw8 = a31 + aw7
aw9 = a32 + aw8
aw10 = -a32 + aw5
aw11 = a21 - a31
aw12 = aw1 - aw11
P1 = aw2
P2 = aw8
P3 = a11
P4 = a32
P5 = a11
P6 = aw6
P7 = aw10
P8 = a33
P9 = aw9
P10 = aw1
P11 = a33
P12 = aw3
P13 = a23
P14 = aw7
P15 = a21
P16 = a31
P17 = a31
P18 = aw0
P19 = aw5
P20 = aw12
P21 = a12
P22 = a22
P23 = aw4
## B-side
bw0 = b13 + b33
bw1 = b11 + b31
bw2 = b11 - b21
bw3 = b11 + b13
bw4 = b23 + b33
bw5 = b13 + bw2
bw6 = bw2 - bw4
bw7 = -b23 + bw5
bw8 = b32 + bw1
bw9 = b12 + bw8
bw10 = -bw6 + bw8
bw11 = b22 + bw10
bw12 = -b21 + bw10
bw13 = -b23 + bw12
Q1 = bw10
Q2 = bw5
Q3 = bw0
Q4 = b22
Q5 = bw9
Q6 = bw4
Q7 = b23
Q8 = b31
```

```

Q9 = b21
Q10 = bw6
Q11 = b32
Q12 = bw13
Q13 = b32
Q14 = bw3
Q15 = b12
Q16 = b12
Q17 = b13
Q18 = bw1
Q19 = bw12
Q20 = bw2
Q21 = bw11
Q22 = b22
Q23 = bw7
## products
M1 = P1 * Q1
M2 = P2 * Q2
M3 = P3 * Q3
M4 = P4 * Q4
M5 = P5 * Q5
M6 = P6 * Q6
M7 = P7 * Q7
M8 = P8 * Q8
M9 = P9 * Q9
M10 = P10 * Q10
M11 = P11 * Q11
M12 = P12 * Q12
M13 = P13 * Q13
M14 = P14 * Q14
M15 = P15 * Q15
M16 = P16 * Q16
M17 = P17 * Q17
M18 = P18 * Q18
M19 = P19 * Q19
M20 = P20 * Q20
M21 = P21 * Q21
M22 = P22 * Q22
M23 = P23 * Q23
## C-side
cw0 = -M19 + M6
cw1 = cw0 + M11
cw2 = M8 + cw1
cw3 = -M12 - cw2
cw4 = -M17 - cw3
cw5 = M2 + cw4
cw6 = M1 + -M13
cw7 = cw6 + M10
cw8 = -M14 + -M12
cw9 = cw8 + cw5
cw10 = M18 + cw7
cw11 = cw2 + cw10
cw12 = M5 + M21
cw13 = cw12 - cw10
cw14 = M3 + cw3
cw15 = cw7 + -M20
cw16 = cw15 + cw9
cw17 = M15 + M22
cw18 = cw17 - -M13
cw19 = M23 - cw5
cw20 = cw19 - M10
cw21 = cw20 - -M20
cw22 = M9 - cw1
cw23 = cw22 + cw9
cw24 = M4 + M16
cw25 = cw24 + M11
cw26 = -M7 + M6
cw27 = cw26 - cw4
C11 = cw11

```

```

C12 = cw13
C13 = cw14
C21 = cw16
C22 = cw18
C23 = cw21
C31 = cw23
C32 = cw25
C33 = cw27

```

## Appendix A1: a complete 56-addition program (class i19w225c4efh)

Exact input sides (13 + 14, both provably minimal for this representative by the side-minimizer's exhaustion), greedy output side (29). On disk: `matmul/external/i19-56adds-slp.txt` with bits at `matmul/external/i19-perminov56.bits` (= Perminov's published cn122 scheme, our sign model m0). Note the *A*-side: one long addition chain — the structure pair extraction cannot express.

```

## A-side: 13 additions
aw0 = a13 + a23      P1 = aw2      P13 = a23
aw1 = a11 + aw0       P2 = aw8      P14 = aw7
aw2 = a12 + aw1       P3 = a11      P15 = a21
aw3 = -a23 + aw2      P4 = a32      P16 = a31
aw4 = a22 + aw2       P5 = a11      P17 = a31
aw5 = a33 + aw3       P6 = aw10     P18 = aw0
aw6 = a32 + aw5       P7 = aw6      P19 = aw5
aw7 = a21 + aw4       P8 = a33      P20 = aw12
aw8 = a31 + aw7       P9 = aw9      P21 = a12
aw9 = a32 + aw8       P10 = aw1     P22 = a22
aw10 = -a12 + aw5     P11 = a33     P23 = aw4
aw11 = a21 + a31      P12 = aw3
aw12 = aw1 + aw11

## B-side: 14 additions
bw0 = b13 - b33      Q1 = bw10     Q13 = b32
bw1 = b11 - b21      Q2 = bw6      Q14 = bw2
bw2 = b11 + b13      Q3 = bw0      Q15 = b12
bw3 = b11 - b31      Q4 = b22      Q16 = b12
bw4 = b23 - b33      Q5 = bw9      Q17 = b13
bw5 = bw1 - bw4      Q6 = bw4      Q18 = bw3
bw6 = b13 + bw1      Q7 = b23      Q19 = bw12
bw7 = -b23 + bw6     Q8 = b31      Q20 = bw1
bw8 = b32 + bw3      Q9 = b21      Q21 = bw11
bw9 = -b12 + bw8     Q10 = bw5     Q22 = b22
bw10 = -bw5 + bw8    Q11 = b32     Q23 = bw7
bw11 = -b22 + bw10   Q12 = bw13
bw12 = -b21 + bw10
bw13 = b23 - bw12

## outputs: 29 additions
cw0 = M11 -M19        C11 = cw4 -cw6
cw1 = M8 -cw0         C12 = -M21 -M5 +M10 -cw6
cw2 = M6 -cw1         C13 = M12 +M3 -cw2
cw3 = M1 -M13         C21 = -cw3 +cw7 +cw8
cw4 = M10 +cw2        C22 = M13 +M15 +M22
cw5 = M2 -M17         C23 = -M12 -M23 +cw5 -cw8
cw6 = M18 -cw3        C31 = M8 +M9 -cw7
cw7 = M14 -cw5        C32 = M11 +M16 +M4
cw8 = M20 -cw4        C33 = -M12 +M17 +M7 -cw1

```

## Appendix B: the side-minimizer on a worked example

Rows (targets) over base variables  $a, b, c, d$ :  $T_1 = a + b$  and  $T_2 = a + b + c + d$ . Both have  $\geq 2$  terms, are distinct up to global sign, and neither is a basis vector, so  $nt = 2$  and any chain needs at least 2 additions.

**Slack**  $h = 0$  (no helpers — every addition must create a target): the pool starts as  $\{a, b, c, d\}$ .  $T_1 = a + b$  is derivable (both operands in the pool): cover it; the pool grows to  $\{a, b, c, d, a+b\}$ . Now  $T_2$  must be  $x \pm y$

with  $x, y$  in the pool: subtracting each pool value from  $T_2$  gives  $c+d, a+c+d, b+c+d, a+b+c, a+b+d$  — none in the pool, so the closure is at a fixpoint with  $T_2$  uncovered, and  $h = 0$  is **exhausted: 2 additions are impossible**.

**Slack  $h = 1$ :** at that same fixpoint the *enabling set* — values  $u$  that would directly complete an uncovered target as  $T_2 = x \pm u$  — is  $\{c+d, a+c+d, b+c+d, a+b+c, a+b+d, \dots\}$ ; of these,  $u = c + d$  is itself derivable from the pool ( $c$  and  $d$  are basis vectors). Insert it as the helper, and the closure resumes:  $T_2 = (a+b) + (c+d)$ .

The optimum is  $nt + h^* = 2 + 1 = \mathbf{3 \text{ additions}}$ , found in 3 search nodes; the emitted chain is

```
w0 = a + b      (covers T1)
w1 = c + d      (helper)
w2 = w0 + w1    (covers T2)
```

replay-verified by expansion. Global signs are free throughout: a row  $-a - b$  is the same target as  $a + b$  (a chain value may be used negated at no cost).

## Appendix C: $\mathbb{Z}$ -scoring a scheme, end to end (sun56)

The pipeline of §4 on the record representative, with every number below reproduced by `sidemin.py --models 24 perminov_cache/bits/sun56.bits`:

1. **Mod-2 gate.** The 621-bit vector satisfies all 729 Brent equations over  $\text{GF}(2)$ .
2. **Sign-SAT.** The support has 175 nonzero coefficients (175 sign variables); the 729 integer Brent equations have 453 covering terms in total, and the instance has 4,269 clauses. Two concrete equations: one type-3 equation is covered by  $k = 3$  terms (the products  $M_2, M_{17}, M_{23}$ ) with right-hand side 1, so **exactly**  $(3-1)/2 = 1$  **of the three terms must be negative**; a neighboring rhs-0 equation is covered by  $k = 4$  terms ( $M_2, M_{17}, M_{20}, M_{23}$ ), so exactly 2 of 4 are negative. kissat solves the instance in milliseconds; each returned model is  $\mathbb{Z}$ -verified (all 729 equations, exact integer arithmetic) before use.
3. **Exact input sides** (model m0). The 23  $A$ -rows contain 12 distinct multi-term targets;  $h = 0$  is exhausted and  $h^* = 1$  succeeds:  $A$ -side =  $12 + 1 = \mathbf{13}$  additions (3 search nodes). The  $B$ -rows contain 11 distinct targets;  $h = 0$  and  $h = 1$  are exhausted,  $h^* = 2$  succeeds:  $B$ -side =  $\mathbf{13}$  (11 nodes). These reproduce Sun's own input-side optima and impossibility certificates.
4. **Greedy output side.** Deterministic pair extraction gives 31 additions — its first extraction is  $w_0 = M_{13} - M_{23}$ , a signed pair shared by 3 of the 9 output forms. Randomized tie-breaking restarts improve this: best over 300 restarts =  $\mathbf{30}$  (the value used throughout; every emitted program is replay-verified).
5. **Total.**  $13 + 13 + 30 = \mathbf{56}$ , on every one of the 24 enumerated sign models.